ACTIVIDAD 5 - GESTIÓN DE PROYECTOS DE INTELIGENCIA ARTIFICIAL

Reporte técnico de AI-SupportAssist

Preparación de datos, comparación de modelos y trazabilidad con MLflow

Estudiante: Efraín Cuatecontzi Mendoza **Matrícula:** AL07180332
Programa: Máster en Inteligencia Artificial **Periodo:** septiembre-diciembre de 2026

Resultado principal y alcance

Se compararon dos clasificadores de intenciones de soporte con 24,014 solicitudes preparadas. LinearSVC obtuvo el mayor F1 macro de validación (0.9919), y un F1 macro de prueba de 0.9927. La diferencia de exactitud frente a Naive Bayes no resultó concluyente al nivel del 5 % (p = 0.0725).

El experimento desarrolla el componente de triaje de la fase 1 de NovaMarket. Utiliza datos sintéticos en inglés y no acredita el 92 % de enrutamiento real, el 45 % de auto-resolución, CSAT ni ROI. La selección experimental requiere validación externa antes de un piloto operativo.

1 Descripción del dataset

Se utiliza *Bitext customer support LLM chatbot training dataset*, de Bitext Innovations (2024), publicado en Hugging Face. Su construcción combina generación sintética y supervisión lingüística. El archivo fijado tiene 26,872 registros, cinco campos, 27 intenciones y 11 categorías observadas. Se prioriza lo verificado en el CSV ante la discrepancia con las diez categorías descritas en parte de la ficha.

Campo original	Tipo y significado	Uso en este estudio
flags	Texto; variaciones lingüísticas	Solo auditoría y contexto
instruction	Texto de la solicitud	Único predictor
category	Texto; grupo semántico	Contexto y validación de etiquetas
intent	Texto; intención etiquetada	Variable objetivo
response	Texto de respuesta de ejemplo	Excluido del predictor para evitar fuga

Supuestos: se consideran utilizables las etiquetas de origen para este benchmark; no se dispone de identificadores de familias de paráfrasis. No se presume representatividad lingüística ni operativa de NovaMarket. La licencia declarada es CDLA Sharing 1.0; se conserva la atribución y la ficha de la revisión fijada. Fuente y diccionario ampliado: datos/README.md.

2 Limpieza y estandarización

La función *preparar_datos* conserva el original, crea una posición source_row y registra la decisión de conservación o exclusión. Las reglas son deterministas y se aplican antes de separar entrenamiento y prueba, para impedir que textos idénticos queden en ambos conjuntos. La auditoría incluye todas las filas, también las descartadas.

Dimensión	Regla aplicada y justificación
Nulos y vacíos	Se leen los cinco campos como texto. Los vacíos en solicitud, intención o categoría se apartan; no se inventan etiquetas ni solicitudes mediante imputación. En el original verificado no se encontraron nulos ni campos vacíos.
Texto	Unicode NFKC, minúsculas y espacios homogéneos. Se conservan acentos y negaciones. No se corrigen automáticamente errores ortográficos porque son parte de las variaciones del benchmark.
Entidades y PII	Las entidades {{...}} pasan al marcador común entity para reducir pistas del tipo de entidad. Correos y teléfonos se sustituyen por marcadores mediante reglas básicas, sin afirmar anonimización completa.
Tipos	instruction, intent, category, response y flags son texto. source_row es entero; text_sha256 es una cadena hexadecimal. Los tipos originales se conservan sin convertir identificadores a cantidades.
Conflictos y duplicados	Se apartan textos normalizados asociados a intenciones incompatibles. Para repeticiones exactas se conserva la primera posición de origen. No se deduplica por cliente o correo.
Codificación categórica	La intención se mantiene como etiqueta textual nominal, compatible con ambos estimadores. category y flags no entran a X; no se impone un orden numérico. La matriz indicadora de 27 clases se usa solo para evaluar ROC-AUC.
Escalado y representación	TF-IDF con frecuencia sublineal, IDF suavizado y normalización L2. No se aplica StandardScaler ni se centra la matriz dispersa. El vocabulario y el IDF se aprenden únicamente dentro del Pipeline de cada fold.

Resultado comprobado de las reglas

Control	Antes	Después
Registros	26,872	24,014
Registros excluidos por repetición	0	2,858
Vacíos en campos originales	0	0
Textos con etiquetas contradictorias	0	0
Textos idénticos entre train y test	No aplica	0

Se excluyeron 2,858 filas (10.64 %). La disminución de filas no equivale a imputación ni a corrección de todos los problemas semánticos. Se conservan errores de escritura y longitudes extremas cuando no existe una regla de negocio suficiente para eliminarlos.

TF-IDF genera características numéricas no negativas. Se usa el tokenizador estándar de palabras de al menos dos caracteres, sin lista de stopwords. Aunque la copia de auditoría conserva signos, la representación de palabras no los utiliza como tokens independientes (Scikit-learn developers, s. f.-a).

3 Estrategia de versionamiento y reproducción

La entrega utiliza scripts de obtención en lugar de subir el CSV completo. El archivo *dataset.lock.json* fija la revisión pública, URL, tamaño y SHA-256. La descarga se valida antes de instalar el archivo definitivo y una copia existente alterada se rechaza sin sobrescribirla. El historial Git conserva la evolución del código y las decisiones en CHANGELOG.md.

Capa	Identificación y control
Origen	Bitext; revisión 430d1a89bd93bd1fa23c16f29dd53e73f0087443. Se comprueban CSV y ficha de origen.
Preparación	preparacion_v1; hashes de tickets_clean_v1.csv, split_v1.csv y folds_v1.csv. Auditoría completa por source_row y texto normalizado.
Particiones	80 % entrenamiento: 19,211 filas. 20 % prueba: 4,803 filas. Estratificación por intención, semilla 42 y cero intersección de huellas de texto.
Código	Git, requirements.txt y SHA-256 de datos_prep.py, experimentos.py y train.py. Los hashes identifican los bytes usados aunque el commit base sea anterior a la ejecución.
Experimentos	MLflow conserva configuración, parámetros, métricas, runs y modelos. Cada reproducción genera run_id nuevos; los resultados numéricos y las particiones se contrastan con las evidencias.
Publicación	Repositorio Act5_GP. Los datos limpios, modelos y base SQLite se regeneran y no se versionan como binarios del repositorio. Las evidencias exportadas sí quedan publicadas.

Huellas de los archivos centrales

CSV original

`6f81102b0100b97b8468eb04368033a23206bf1fde9d53500d5806ec1001a434`

Dataset preparado

`bf4da7cdace69cfa7ffb3a4c138ea98f215baf42a9728f4123152c6d30c41789`

Partición train/test

`bc9bce39ee2e08d2d734c6d6d092ebe668e3b1bb97933f39101684bb07432383`

Folds de validación

`f9709d2766d6c4e8f9309912fa270b37093704d983a11748ac750a3b0268ecc9`

Commit que publica código y evidencias de entrenamiento:
2890e1d1a46da9401c8bc7d006d0c507a415d1d4

La ejecución registra como commit base 4818abf, correspondiente al estado previo de la etapa. La coincidencia de los hashes de las fuentes verifica el código incorporado al commit de entrenamiento. No se afirma que los identificadores de MLflow existan en una instalación nueva.

Control de cambios

Versiones documentadas: 0.1.0 estructura del repositorio; 0.2.0 configuración de MLflow; 0.3.0 obtención verificada; 0.4.0 preparación, entrenamiento y evaluación. Una modificación de las reglas, del origen o de la partición requiere una versión nueva y un experimento identificable, preservando el historial previo.

4 Visualizaciones y análisis de calidad

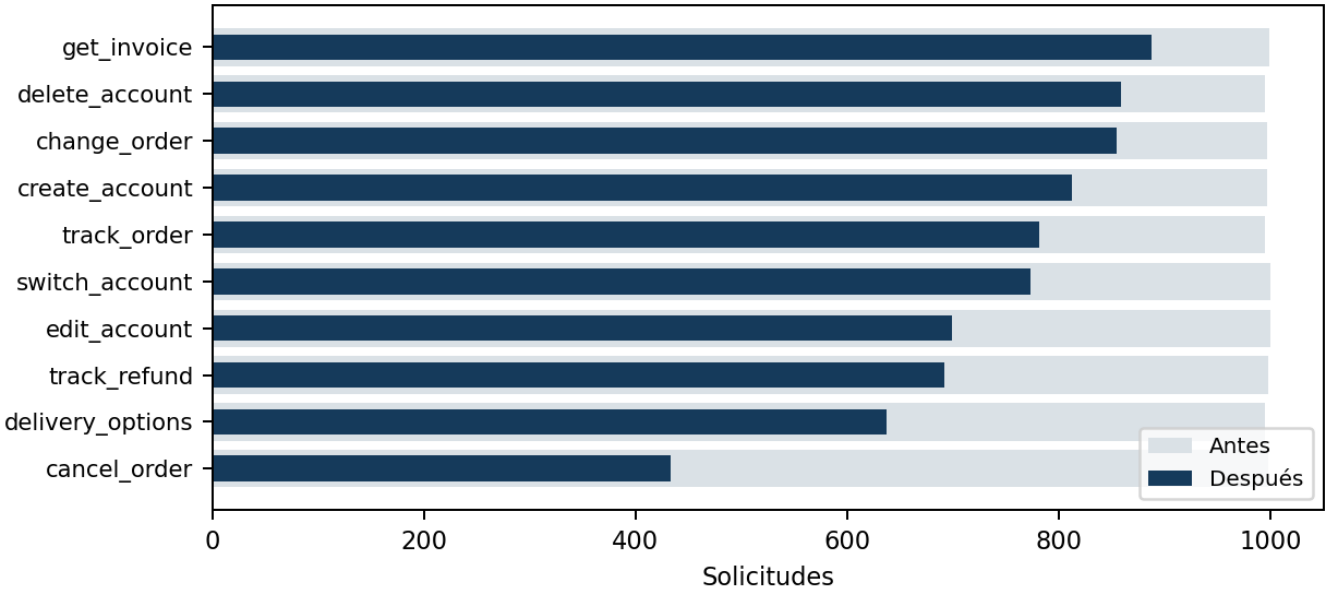


Figura 1. Diez intenciones con menos solicitudes después de la deduplicación. Se comparan los conteos originales con los preparados. Fuente: ejecución propia sobre Bitext.

El original era casi uniforme por intención, pero la repetición afectaba de forma desigual a las clases. cancel_order conserva 433 solicitudes de 998; complaint conserva 997. La razón entre la clase mayor y menor después de limpiar es 2.30. Este cambio justifica estratificar y priorizar F1 macro, sin sobremuestrear ni volver a introducir duplicados.

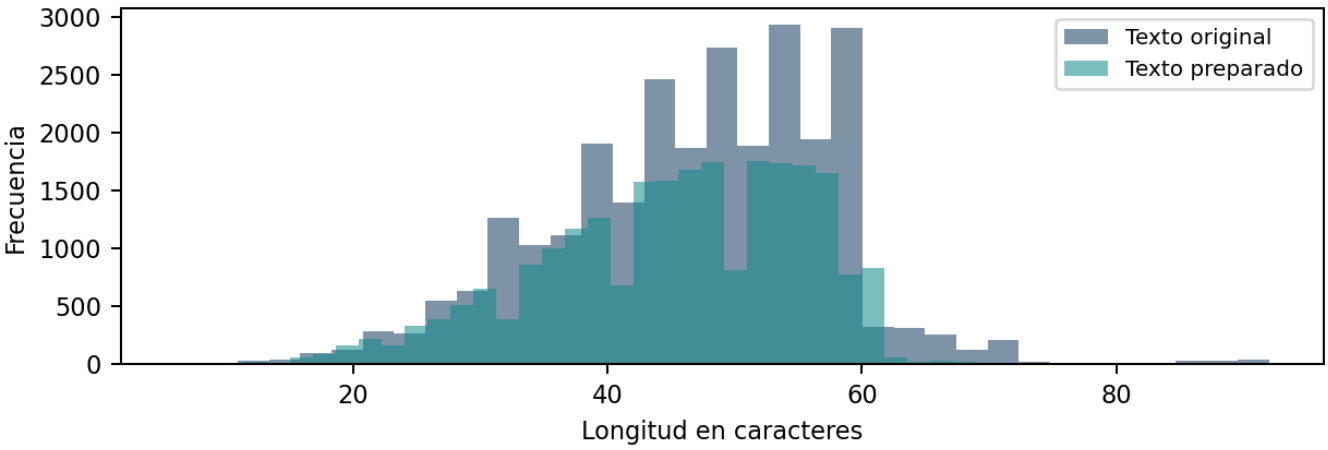


Figura 2. Distribución de longitudes antes y después de preparar. La sustitución de entidades y la deduplicación modifican la forma del histograma; esto no implica una reducción equivalente de complejidad semántica.

Los textos preparados tienen media 45.23, mediana 46 y rango de 6 a 69 caracteres. La brevedad y concentración son compatibles con un benchmark generado. Los extremos se revisan y se conservan; no hay evidencia para declararlos errores. La eliminación de coincidencias exactas no garantiza independencia entre paráfrasis.

5 Modelos entrenados

Multinomial Naive Bayes: referencia probabilística de baja complejidad, apropiada para características textuales no negativas. El suavizado alpha reduce problemas de frecuencias raras. Se usa TF-IDF como aproximación práctica; la independencia condicional de términos es un supuesto simplificador, especialmente con bigramas (Scikit-learn developers, s. f.-b).

SVM lineal (LinearSVC): clasificador de margen máximo regularizado, aplicado uno contra el resto. Admite matrices dispersas y modela fronteras lineales de forma eficiente. Sus márgenes permiten ordenar ejemplos, pero no son probabilidades calibradas. Se fija random_state=42 y max_iter=10,000 (Scikit-learn developers, s. f.-c).

Ambos reciben únicamente text_clean. El Pipeline guarda vectorizador y clasificador juntos para mantener la transformación usada en entrenamiento durante inferencia. Como referencia trivial adicional se utiliza DummyClassifier de clase mayoritaria; no participa en la selección.

6 Configuración de hiperparámetros

Configuración	Espacio evaluado	Selección por CV
TF-IDF común	ngram_range: (1,1), (1,2); min_df=2; max_features=20,000; sublinear_tf=True; norm=L2	Bigramas en ambos modelos
MultinomialNB	alpha: 0.1, 0.5, 1.0	alpha=0.1
LinearSVC	C: 0.5, 1.0, 2.0	C=1.0
Búsqueda	6 combinaciones por modelo; StratifiedKFold de 5 folds; misma partición de folds	60 ajustes de CV en total y 2 reajustes finales

La cuadrícula explora suavizado o regularización y el valor de agregar contexto de dos palabras. Se limita el vocabulario y la frecuencia mínima para reducir ruido y costo. La búsqueda maximiza F1 macro promedio; accuracy se registra como métrica secundaria. El conjunto reservado de prueba no define los hiperparámetros.

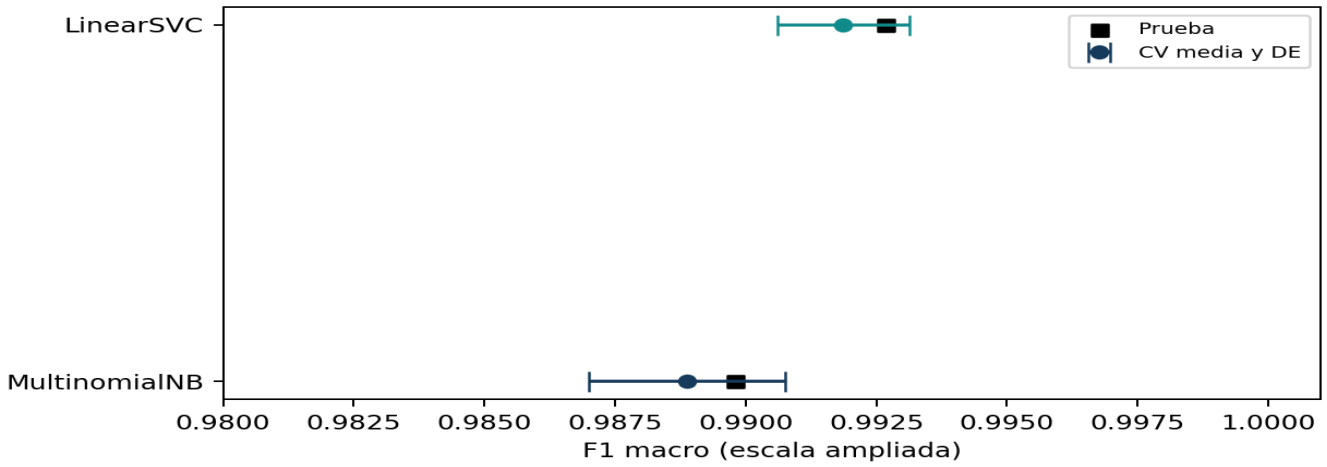


Figura 3. Media y desviación estándar del F1 macro de cinco folds, junto con F1 de prueba. La escala horizontal está ampliada de 0.98 a 1.001 para hacer visibles diferencias pequeñas. Las barras de dispersión no son intervalos de confianza.

7 Métricas obtenidas e interpretación

La exactitud es la fracción de aciertos. Precision y recall se calculan por intención y después se promedian sin ponderar; F1 macro promedia la media armónica por clase. Así cada intención aporta igual peso. ROC-AUC se calcula para cada clase frente al resto y se promedian las 27 áreas; no se calcula a partir de etiquetas predichas (Scikit-learn developers, s. f.-d).

Métrica en prueba	MultinomialNB	LinearSVC
Accuracy	0.9904	0.9931
Precision macro	0.9904	0.9934
Recall macro	0.9893	0.9921
F1 macro	0.9898	0.9927
ROC-AUC macro OvR	0.999988	0.999981
Errores / 4,803 solicitudes	46	33
F1 CV media y DE	0.9889 / 0.0019	0.9919 / 0.0013

El baseline obtiene accuracy 0.0414 y F1 macro 0.0029. Los dos modelos lo superan ampliamente. SVM mejora F1 macro, mientras Naive Bayes tiene un ROC-AUC ligeramente mayor. No son conclusiones contradictorias: F1 evalúa decisiones de clase y AUC evalúa ordenamiento de puntuaciones. El AUC elevado no demuestra calibración ni seguridad de auto-resolución.

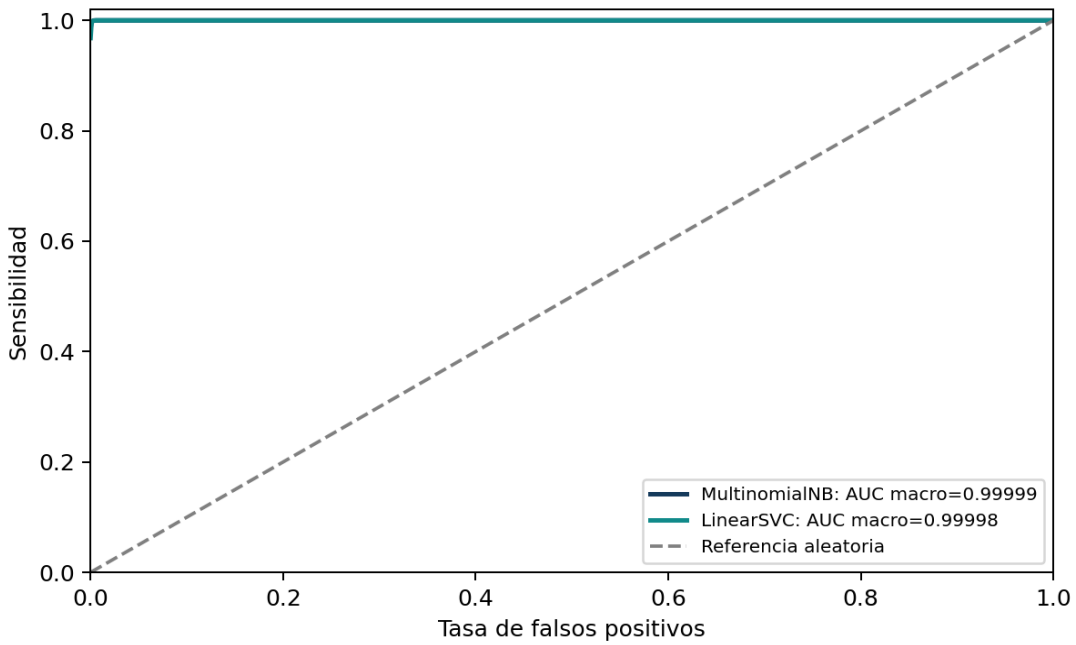


Figura 4. Curva ROC media interpolada para visualización. La leyenda muestra la media de áreas calculadas por clase. Naive Bayes usa predict_proba y SVM usa decision_function en 27 evaluaciones binarias; no se aplica softmax para fabricar probabilidades.

MAE, RMSE y R^2 no aplican: el objetivo es una intención nominal, no una variable continua. Las cifras altas corresponden a un benchmark sintético con solicitudes cortas y no se extrapolan a soporte real en español.

8 Comparación estadística de resultados

Se usa McNemar exacta sobre los mismos 4,803 tickets de prueba. La hipótesis nula establece igual probabilidad de acierto entre modelos. Condicionada a los pares discordantes, se evalúa una binomial bilateral con probabilidad 0.5. El contraste utiliza accuracy, no F1 macro, y fija alfa=0.05. No se aplica una prueba t ingenua a folds que comparten datos de entrenamiento (SciPy developers, s. f.).

Resultado pareado	SVM correcto	SVM incorrecto
Naive Bayes correcto	4741	16
Naive Bayes incorrecto	29	17

Hay 45 pares discordantes: 16 favorecen a Naive Bayes y 29 a SVM. Se obtiene **p=0.072454**. No se rechaza la igualdad de exactitud al 5 %. Esto no demuestra equivalencia; la evidencia no permite afirmar superioridad estadística en este contraste.

Un bootstrap pareado de 2,000 remuestreos con semilla 42 estima la diferencia accuracy(NB)-accuracy(SVM): **-0.2707 puntos porcentuales**, con IC percentil del 95 % **[-0.5205; 0.0000]**. El intervalo incluye cero. El remuestreo se realiza por ticket y presupone independencia; las posibles familias de paráfrasis hacen que la lectura sea exploratoria.

Errores que requieren atención

Intención real en SVM	Recall	Soporte de prueba
cancel_order	0.9425	87
get_invoice	0.9719	178
registration_problems	0.9747	198
create_account	0.9815	162

SVM confunde cinco cancelaciones con cambios de pedido, cinco problemas de registro con incidencias de pago y cinco solicitudes de obtener factura con consultar factura. Son distinciones de negocio que podrían causar enrutamientos inadecuados. La selección por CV no elimina la necesidad de revisión por clase y validación humana.

Decisión técnica y recursos

Recurso observado	Naive Bayes	SVM lineal
Tiempo de búsqueda CV (s)	6.08	11.89
Latencia p95 (ms / solicitud)	0.545	0.295
Pipeline serializado (MiB)	2.23	1.21

Se recomienda SVM para continuar el prototipo por su mayor F1 macro de CV, menor tamaño y menor latencia observada. Naive Bayes conserva valor como baseline de entrenamiento más rápido. Los tiempos corresponden a CPU local y 200 solicitudes después de calentamiento; excluyen red, CRM, LLM y RAG.

9 Evidencia del uso de MLflow

El seguimiento utiliza MLflow 3.5.0, base SQLite y almacén de artefactos local. El experimento **Act5_GP_Entrenamiento_v1** es distinto de la prueba de configuración del paso 2. La evidencia del reporte procede de 15 runs finalizadas: dos padres de modelos, doce hijos de candidatos y una comparación estadística (MLflow Project, s. f.).

Modelo	Run ID de entrenamiento
MultinomialNB	992ed45a354740febfd30cf142c38c83
LinearSVC	07cca257012f4410b5a3c58e0cb2e8ae

Elemento registrado	Evidencia comprobada
Parámetros	Semilla, folds, scoring, reglas TF-IDF, tamaño de particiones, hashes y mejores hiperparámetros. Cada candidato conserva sus parámetros y F1 por fold.
Métricas	F1 de validación y dispersión, accuracy, precision/recall/F1 macro, ROC-AUC, F1 de entrenamiento, latencia, tiempo y tamaño.
Artefactos	Manifiesto de preparación, entorno, diagnóstico, tablas Grid Search, métricas por clase y gráficas. El run estadístico registra las evidencias comparativas.
Modelos	Se registra el Pipeline completo, firma y ejemplo de entrada. Se recarga desde MLflow y se comprueba igualdad de todas sus predicciones de prueba.
Persistencia	Un cliente independiente consulta runs, estados, parámetros, métricas y listado de artefactos; se contrasta con los CSV y JSON exportados.

Fragmento de la evidencia recuperada

MultinomialNB: estado FINISHED; F1 macro=0.98980192; ROC-AUC=0.99998799; modelo_mlflow_recuperado=True.

LinearSVC: estado FINISHED; F1 macro=0.99267755; ROC-AUC=0.99998132; modelo_mlflow_recuperado=True.

Verificaciones: 15 estados FINISHED; métricas F1 y ROC-AUC recuperadas por API; manifiesto presente; ambos modelos recargados desde sus URI runs:/ y predicciones verificadas. Las fuentes y artefactos de configuración anteriores conservan su propio run histórico.

Archivos auditables en el repositorio

evidencias/entrenamiento_v1/ contiene mlflow_evidencia.json, mlflow_runs.csv, comparacion_modelos.csv, resultados por clase y estadistica.json. Los hashes de las fuentes se incluyen en datos/preparacion_v1.json.

La base y los binarios se regeneran al ejecutar el proyecto. Una exportación de JSON no equivale a trasladar la base de MLflow: cada usuario obtendrá nuevos identificadores en su entorno. En Colab se puede conservar el tracking en Drive. Esta entrega no afirma haber operado un servidor compartido ni desplegado los modelos.

10 Deuda técnica y enfoque MLOps

El código de aprendizaje representa una parte del sistema. Los datos, las configuraciones y las dependencias también generan obligaciones de mantenimiento. La trazabilidad reduce ambigüedad sobre qué se ejecutó, pero no elimina la deuda de cobertura del dominio ni garantiza estabilidad operativa (Sculley et al., 2015).

Deuda o riesgo	Acción propuesta	Responsable e indicador
Idioma y dominio	Construir evaluación externa con tickets reales anonimizados en español, etiquetas revisadas y periodos posteriores.	Responsable de soporte y ciencia de datos; F1 y recall por intención e idioma.
Paráfrasis y plantillas	Obtener identificadores de familias y evaluar por grupos; deduplicación aproximada revisada por especialistas.	Ingeniería de datos; intersección de familias y caída del desempeño externo.
Configuración y dependencias	Mantener versiones fijadas, hashes, pipelines completos y pruebas de recarga. Revisar actualizaciones antes de promoverlas.	MLOps; reproducción, integridad y compatibilidad.
Drift y nuevas clases	Monitorear distribución de intenciones y errores con etiquetas humanas; revisar y reentrenar con aprobación.	MLOps y soporte; exactitud real inferior al 90 % como alerta inicial de fase 1.
Privacidad y seguridad	Ampliar anonimización, control de acceso y auditoría de ejemplos antes de usar datos reales.	Seguridad y responsable de datos; incidentes y tasa de hallazgos en muestras.
Abstención y supervisión	Calibrar o validar umbrales en un conjunto distinto; escalar casos sensibles o desconocidos.	Líder de soporte; error y cobertura de las solicitudes aceptadas.

Reflexión técnica

La principal deuda pendiente es de validez externa. Un resultado cercano a uno en un conjunto sintético puede coexistir con fallas en lenguaje espontáneo, solicitudes con varias intenciones o políticas específicas del negocio. Por ello, registrar más experimentos no sustituye mejorar la evidencia de datos. El siguiente hito debe ser una evaluación externa que permita medir el costo de los errores de enrutamiento y la necesidad de intervención humana.

El diseño modular permite cambiar el clasificador conservando la limpieza y el contrato de inferencia. Sin embargo, cualquier cambio de reglas o etiquetas exige revisar el historial y la partición. La promoción de un modelo debe requerir calidad, seguridad y capacidad operativa, con posibilidad de volver a una versión anterior. Los umbrales de fase 1 son criterios iniciales de gobierno, no garantías demostradas por este benchmark.

Conclusión

La actividad integró obtención verificable, preparación documentada, comparación con validación cruzada y seguimiento de modelos. De 26,872 filas se conservaron 24,014; se eligió LinearSVC por F1 macro de CV. Su ventaja de exactitud en prueba no fue concluyente al 5 %. La recomendación es continuar con un piloto de evaluación controlado, manteniendo supervisión humana y verificando desempeño en español antes de actualizar las proyecciones financieras de AI-SupportAssist.

Referencias

Bitext Innovations. (2024). *Bitext customer support LLM chatbot training dataset* [Conjunto de datos]. Hugging Face. https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset

MLflow Project. (s. f.). *MLflow Tracking Quickstart*. https://mlflow.org/docs/latest/ml/tracking/quickstart/

Scikit-learn developers. (s. f.-a). *Feature extraction*. https://scikit-learn.org/stable/modules/feature_extraction.html

Scikit-learn developers. (s. f.-b). *MultinomialNB*. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html

Scikit-learn developers. (s. f.-c). *LinearSVC*. https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html

Scikit-learn developers. (s. f.-d). *roc_auc_score*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html

SciPy developers. (s. f.). *scipy.stats.binomtest*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.binomtest.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems, 28*, 2503-2511. https://research.google/pubs/hidden-technical-debt-in-machine-learning-systems/

Reproducción y evidencia de la entrega

Repositorio: https://github.com/efrainkun-ai/Act5_GP. Ejecutar desde su raíz con Python 3.12; CPU es suficiente. Los tiempos y los run_id pueden variar entre entornos.

```
python -m pip install -r requirements.txt
```

```
python fuentes/obtener_datos.py
```

```
python fuentes/train.py --entrenar
```

Versiones verificadas: numpy 2.2.6, pandas 2.3.3, scikit-learn 1.7.2, scipy 1.16.3, matplotlib 3.10.7, mlflow 3.5.0, joblib 1.5.2. SQLAlchemy 2.0.43 y packaging 25.0 se fijan también en requirements.txt.

La fuente fija y los hashes están en datos/dataset.lock.json y datos/preparacion_v1.json. Los datos limpios y la auditoría se regeneran en datos/datos_limp/. Los resultados por clase, CV y estadística se conservan en evidencias/entrenamiento_v1/. La corrida de entrenamiento se realizó en entorno local; el notebook de Colab reproduce el mismo script.

El informe se basa en el dataset público y en una ejecución nueva del proyecto. Los resultados financieros y operativos de la fase 1 se mantienen como supuestos. No se presentan como mediciones derivadas del experimento.